Addis Ababa University
አዲስ አበባ ዩኒቨርሲቲ
SEEK WISDOM, ELEVATE YOUR INTELLECT AND SERVE HUMANITY !

Course :Introduction to Information Storage and Retrieval

1. Kaleab Abebe……...UGR/4485/17
2. Kaleb Jemberu……UGR/8686/17
3. Noel Anteneh……UGR/8132/17
4. Samuel Ashenafi…….UGR/1090/17
5. Samuel Yeshambel…….UGR/5294/17
6. Yeamlak Sisay……..UGR/8919/17

Instructor:Demeke A. Ayele

INFORMATION RETRIEVAL

Cranfield Collection Evaluation Report

1. Introduction

Information Retrieval (IR) is the discipline concerned with finding material — usually documents — from large collections that satisfies an information need. The effectiveness of an IR system depends critically on the ranking model it uses: the algorithm that scores each document for a given query and determines the order in which results are presented.

This report presents a systematic evaluation of five classical ranking models implemented against the Cranfield collection — one of the oldest and most widely used benchmark test collections in IR research. The Cranfield collection was compiled at Cranfield University in the 1960s and consists of 1,400 aeronautics research abstracts, 225 queries, and 1,612 manually assessed relevance judgements. Despite its age, it remains a standard evaluation benchmark in the IR community.

The five ranking models evaluated are:

- TF-IDF — the classical term frequency / inverse document frequency weighted model
- Okapi TF — a length-normalised term frequency model without an IDF component
- BM25 (Okapi BM25) — the probabilistic best-match model combining Robertson IDF with length-normalised TF
- Laplace Language Model — a query-likelihood model using add-1 (Laplace) smoothing
- Jelinek-Mercer Language Model — a query-likelihood model using linear interpolation smoothing

The system is implemented in Python 3.10 with Elasticsearch 9.3.3 as the document store and term vector provider. All five models are implemented manually from first principles — Elasticsearch is used only for storage and term statistics, not for built-in scoring. Evaluation metrics including MAP, Precision@10, Precision@20, Recall@100, and NDCG@10 were computed using a custom Python evaluation script.

The remainder of this report is structured as follows: Section 2 provides background on the Cranfield paradigm and evaluation metrics; Section 3 describes the system architecture; Section 4 details each ranking model; Section 5 presents the experimental results; Section 6 analyses and discusses the findings; Section 7 concludes; and Section 8 lists references.

2. Background & Related Concepts

2.1 The Cranfield Evaluation Paradigm

The Cranfield evaluation methodology, pioneered by Cyril Cleverdon in the 1960s, established the use of a fixed document collection, a set of queries, and relevance judgements (qrels) as the foundation for comparing IR systems in a controlled, reproducible manner. This test-collection-based approach remains the dominant evaluation methodology in IR, forming the basis of the TREC (Text Retrieval Conference) campaigns that have driven IR research since 1992.

The key properties of the Cranfield collection used in this study are:

- 1,400 documents: short aeronautics research abstracts, typically between 50 and 300 tokens in length
- 225 queries: technical questions about aerodynamics, heat transfer, boundary layers, fluid mechanics, and related topics
- 1,612 relevance judgements: an average of 7.16 relevant documents per query, assessed on a graded relevance scale
- Average document length: 173.72 tokens; vocabulary size: 7,471 distinct terms; total collection tokens: 243,204

2.2 Evaluation Metrics

The following metrics are used to compare model performance in this study:

Mean Average Precision (MAP): The primary metric. For each query, Average Precision (AP) is computed as the mean of precision values at each rank position where a relevant document is retrieved. MAP is the mean of AP scores over all queries. It rewards both high precision and early retrieval of relevant documents, making it a comprehensive single-number summary of ranking quality.

Precision at k (P@k): The fraction of the top-k retrieved documents that are relevant. P@10 reflects the quality of the first page of results as a user would experience it. P@20 extends this to a second page.

Recall at 100 (R@100): The fraction of all relevant documents that appear in the top 100 retrieved results. This measures the system's ability to comprehensively cover the relevant document set within a practical retrieval depth.

Normalised Discounted Cumulative Gain at 10 (NDCG@10): A rank-aware metric that assigns higher gain to relevant documents appearing earlier in the ranking. The discounting is logarithmic (1/log2(rank+1)). The score is normalised by the ideal DCG achievable for the query, making it comparable across queries with different numbers of relevant documents.

3. System Architecture & Implementation

3.1 Pipeline Overview

The IR pipeline is implemented across six Python modules, each corresponding to a distinct step in the workflow. Elasticsearch 9.3.3 serves as the document store, providing persistent storage and efficient term vector retrieval.

Module	File	Responsibility
Parser	parser.py	Parses XML documents, XML queries, and TREC-format qrels
ES Setup	elasticsearch_setup.py	Creates Elasticsearch index with term vectors; bulk-indexes 1,400 documents
Helpers	helpers.py	Bulk-fetches all term vectors; exposes TF, DF, length, and collection stats
Models	models.py	Implements all five scoring functions from mathematical definitions
Ranker	ranker.py	Scores all 1,400 docs per query per model; writes TREC-format result files
Evaluator	evaluator.py	Loads results files; computes and reports all evaluation metrics

3.2 Elasticsearch Index Design

Documents are indexed with the body_text field configured with term_vector: "with_positions_offsets". This enables retrieval of per-document term frequency statistics through Elasticsearch's mtermvectors API without requiring a separate inverted index implementation. The index uses a single shard with no replicas, appropriate for a local development deployment on a fixed collection.

3.3 Performance Optimisation: Bulk Term Vector Loading

A critical design decision was the batch retrieval of term vectors. The naive approach — issuing one Elasticsearch request per (term, document) pair — would require up to 1.5 million network round-trips for the full pipeline (1,400 documents x 225 queries x 5 models). This rendered the system impractically slow in initial testing, taking several hours to complete.

The optimised solution issues a single mtermvectors request at startup that retrieves all 1,400 document term vectors simultaneously. The response is parsed into in-memory Python dictionaries, and all subsequent scoring operations access these dictionaries directly without any further Elasticsearch communication. This reduces total pipeline execution time from several hours to approximately 60 seconds.

3.4 Output Format

Results are written in standard TREC format:

```
query-id Q0 docno rank score model_name
```

Each model produces a separate output file (e.g. bm25.txt, tfidf.txt) in the results/ directory. These files are compatible with the trec_eval evaluation tool used in TREC campaigns.

4. Ranking Models

This section presents the mathematical formulation of each ranking model, the parameter settings used, and the theoretical motivation behind each approach. All scoring functions take a list of query terms and a document ID as input and return a float relevance score.

4.1 TF-IDF

TF-IDF (Term Frequency-Inverse Document Frequency) is the foundational vector space model for document ranking. The score of document d for query q is:

$$\text{score}(q, d) = \Sigma_{t \in q} \ \text{tf}(t, d) \times \log(N / \text{df}(t))$$

where tf(t,d) is the raw count of term t in document d, N = 1,400 is the total number of documents, and df(t) is the number of documents containing term t. The log(N/df) term is the IDF weight, which assigns high values to rare, discriminative terms and low values to common terms.

TF-IDF does not normalise for document length, meaning that longer documents will systematically accumulate higher TF values and thus higher scores, independent of their actual relevance to the query.

4.2 Okapi TF

Okapi TF addresses the length normalisation problem by dividing TF by a length-dependent factor, but omits the IDF component entirely:

$$\text{tf_norm}(t, d) = \text{tf}(t, d) / (\text{tf}(t, d) + k1 \times (1 - b + b \times dl / \text{avgdl}))$$

where k1 = 1.2 is the TF saturation parameter, b = 0.75 controls the degree of length normalisation, dl is the document length in tokens, and avgdl = 173.72 is the mean document length. The final score is the sum of tf_norm values over query terms.

The b parameter creates a spectrum: b = 0 gives no length normalisation (raw TF), while b = 1 gives full length normalisation (tf relative to document length). The commonly used b = 0.75 provides partial normalisation.

4.3 BM25 (Okapi BM25)

BM25 is the most complete probabilistic model in this study, combining Robertson IDF with length-normalised TF and a query-term frequency component. It is the state-of-the-art baseline for keyword-based IR:

$$\text{score}(q, d) = \Sigma_{t \in q} \ \text{IDF}(t) \times \text{tf_norm}(t, d) \times \text{qtf_weight}$$

$$\text{IDF}(t) = \log((N - \text{df}(t) + 0.5) / (\text{df}(t) + 0.5))$$

$$\text{tf_norm}(t, d) = \text{tf}(t, d) \times (k1 + 1) / (\text{tf}(t, d) + k1 \times (1 - b + b \times dl / \text{avgdl}))$$

$$\text{qtf_weight} = (k3 + 1) \times \text{qtf} / (k3 + \text{qtf})$$

Parameters used: k1 = 1.5, b = 0.75, k3 = 500. The Robertson IDF formula smooths the classic log(N/df) to avoid negative values when df > N/2. TF saturation means the marginal benefit of each additional term occurrence decreases — a term appearing 10 times contributes far less than 10x

the score of a term appearing once. The qtf component handles repeated query terms, though it has minimal effect for single-occurrence query terms (qtf=1).

4.4 Laplace Language Model

Language models approach retrieval as a probability estimation problem: given the query as an observed sequence of terms, score document d by estimating the probability that d's language model would generate those terms. Laplace smoothing prevents zero probabilities for terms absent from a document:

```
P(t | d) = (tf(t, d) + 1) / (dl(d) + V)

score(q, d) = Σ_{t ∈ q}  log P(t | d)
```

where V = 7,471 is the vocabulary size. The smoothed probability assigns a minimum probability of 1/(dl+V) to every term in the vocabulary, regardless of whether it appears in the document. Log-probabilities are summed to avoid floating-point underflow with small probabilities.

4.5 Jelinek-Mercer Language Model

Jelinek-Mercer smoothing replaces the uniform smoothing mass of Laplace with a mixture of the document model and the collection-level language model:

```
P(t | d) = (1 - λ) × tf(t, d) / dl(d)  +  λ × ctf(t) / C

score(q, d) = Σ_{t ∈ q}  log P(t | d)
```

where lambda = 0.4 is the interpolation parameter, ctf(t) is the total collection frequency of term t across all documents, and C = 243,204 is the total number of tokens in the collection. When a term is absent from a document (tf=0), the score falls back to lambda x P(t|C) rather than a fixed constant, so common collection terms receive a higher floor probability than rare ones.

5. Results

5.1 Overall Performance Comparison

Table 1 presents the evaluation metrics for all five models evaluated over 225 queries on the Cranfield collection. Models are sorted by MAP in descending order. The BM25 row is highlighted to indicate the best-performing system.

Model	MAP	P@10	P@20	R@100	NDCG@10
BM25	**0.2750**	**0.2276**	**0.1531**	**0.7063**	**0.3647**
Jelinek-Mercer LM	0.2444	0.2013	0.1331	0.6721	0.3307
TF-IDF	0.2169	0.1756	0.1260	0.6701	0.2890
Okapi TF	0.2002	0.1769	0.1207	0.5987	0.2773
Laplace LM	0.1070	0.0956	0.0716	0.4324	0.1572

Table 1: Overall evaluation metrics. Models sorted by MAP (descending). BM25 row highlighted.

BM25 achieves the highest performance across all five metrics: MAP = 0.2750, P@10 = 0.2276, P@20 = 0.1531, R@100 = 0.7063, and NDCG@10 = 0.3647. The Jelinek-Mercer language model is second (MAP = 0.2444), followed by TF-IDF (MAP = 0.2169), Okapi TF (MAP = 0.2002), and Laplace LM (MAP = 0.1070).

The most striking result is the poor performance of the Laplace language model. Its MAP of 0.1070 is 61.1% lower than BM25 and 56.2% lower than Jelinek-Mercer — its closest language model competitor. This large gap between two models with similar theoretical foundations underscores the critical importance of the smoothing strategy.

The R@100 metric reveals that BM25 retrieves 70.63% of all relevant documents within its top 100 results, substantially more than Laplace (43.24%). This suggests that the Laplace model not only ranks relevant documents lower but fails to retrieve a significant fraction of them even within 100 results.

5.2 Per-Query Average Precision

Table 2 shows Average Precision scores for each model on the first ten queries. The best-performing model per query is highlighted. This view reveals important patterns in per-query behaviour that aggregate MAP scores can obscure.

QID	TF-IDF	Okapi TF	BM25	Laplace	JM
1	0.1479	0.1636	**0.2081**	0.1062	0.1774
2	**0.1608**	0.1195	0.1485	0.0811	0.1298
3	0.4787	0.3688	**0.6176**	0.0282	0.5537

QID	TF-IDF	Okapi TF	BM25	Laplace	JM
4	**0.6667**	0.5526	0.5909	0.1000	0.5833
5	0.1875	0.1267	**0.1975**	0.0794	0.1436
6	0.0952	**0.1647**	0.1404	0.0860	0.1403
7	0.1462	0.1845	0.1967	0.1127	**0.2708**
8	**0.1748**	0.1472	0.1522	0.1121	0.1312
9	0.2407	**0.8056**	**0.8056**	0.2518	0.7000
10	0.0329	0.0651	**0.1072**	0.0125	0.1043

Table 2: Average Precision per query for queries 1-10. Highlighted values indicate the best model per query.

Several notable patterns emerge from Table 2:

- Query 9 produces exceptionally high AP for BM25 (0.8056) and Okapi TF (0.8056), indicating that the relevant documents for this query contain highly distinctive vocabulary not shared with irrelevant documents. Even models without full BM25 components can easily identify them.
- Query 3 shows the largest inter-model spread: BM25 achieves AP = 0.6176 while Laplace achieves only 0.0282, a difference of 0.5894. This represents a query where collection-level IDF discrimination is crucial.
- Query 4 is the only query in this sample where TF-IDF (AP = 0.6667) outperforms BM25 (AP = 0.5909). This may occur when the relevant documents for a query are particularly long, making the absence of length normalisation in TF-IDF accidentally beneficial.
- Laplace consistently underperforms across all ten sample queries, with its best result being query 9 (AP = 0.2518) — where even a weak model scores well due to the distinctive vocabulary of the relevant documents.

5.3 Top-10 Ranking Comparison

Tables 3-5 show the top-10 ranked documents for three representative queries. Relevant documents are highlighted and marked with *. These tables illustrate how the different scoring strategies lead to different document orderings, particularly in the middle ranks.

Query 1: Relevant documents — 184, 13, 51, 14, 875, 12

#	TF-IDF	Okapi TF	BM25	Laplace	JM
1	1268	184 *	184 *	1268	184 *
2	13 *	1268	486	14 *	486
3	486	486	13 *	792	1268
4	184 *	51 *	12 *	184 *	13 *
5	51 *	12 *	1268	51 *	12 *
6	792	13 *	51 *	1144	51 *
7	1144	14 *	875 *	1147	14 *
8	12 *	588	878	576	878
9	14 *	172	746	588	792
10	686	878	792	12 *	1361

*Table 3: Top-10 results for Query 1. * indicates relevant document.*

BM25 places the most relevant document (184) at rank 1 and retrieves 5 of the 6 relevant documents within its top 10. TF-IDF ranks irrelevant document 1268 first — a longer abstract that accumulates high raw TF but is not relevant. Laplace also places 1268 first and retrieves only 3 relevant documents in the top 10.

Query 5: Relevant documents — 1296, 401

#	TF-IDF	Okapi TF	BM25	Laplace	JM
1	329	103	103	329	103
2	103	1272	1032	540	1032
3	1066	540	1296 *	77	943
4	1296 *	172	943	1248	1296 *
5	14	1296 *	746	1147	828
6	540	746	1272	1201	746
7	401 *	28	650	625	625
8	172	1295	1379	1392	1272
9	1392	625	625	172	28
10	746	329	28	101	540

*Table 4: Top-10 results for Query 5. * indicates relevant document.*

Query 5 has only two relevant documents (1296 and 401). BM25 retrieves 1296 at rank 3, while TF-IDF retrieves both (1296 at rank 4, 401 at rank 7). Laplace fails to retrieve either relevant document in its top 10. JM retrieves 1296 at rank 4 but misses 401.

Query 10: Relevant documents — 302, 405

#	TF-IDF	Okapi TF	BM25	Laplace	JM
1	493	493	493	493	493
2	185	**302 ***	**302 ***	1313	**302 ***
3	1199	1214	949	1199	1286
4	691	1199	1199	576	691
5	**302 ***	1264	524	1214	1214
6	1214	691	1286	691	949
7	1319	1009	691	110	524
8	1274	1286	1010	329	1264
9	332	541	**405 ***	185	1199
10	949	332	1214	**302 ***	781

*Table 5: Top-10 results for Query 10. * indicates relevant document.*

All five models agree that document 493 is the top result for Query 10, demonstrating strong cross-model consensus on the most relevant document. BM25 uniquely retrieves both relevant documents (302 at rank 2, 405 at rank 9), while TF-IDF retrieves only 302 at rank 5. Laplace retrieves 302 only at rank 10.

6. Analysis & Discussion

6.1 Why BM25 Performs Best

BM25 achieves the highest performance across all evaluated metrics. This outcome is consistent with the broader IR literature — BM25 has been the de facto standard ranking function for over two decades and excels particularly on short, technical document collections with precise queries.

The Cranfield collection is well-suited to BM25's strengths for three specific reasons:

Robertson IDF discriminates on specialised vocabulary: Aeronautics research uses highly specific terminology (e.g. 'ablative cooling', 'Mach number', 'laminar-turbulent transition') that appears in only a handful of documents. Robertson IDF assigns these terms very high weights, enabling BM25 to pinpoint the small relevant subset. Common aeronautics terms like 'pressure' and 'flow' appear across hundreds of abstracts and receive low IDF weights, preventing them from dominating scores.

Length normalisation prevents systematic bias: Cranfield abstracts range from approximately 30 to over 400 tokens. Without normalisation, longer abstracts would systematically outscore shorter ones by accumulating more raw TF. The $b = 0.75$ parameter provides partial normalisation, ensuring that a focused 80-token abstract about a highly specific topic can outrank a 400-token abstract that contains the same terms more frequently but with less focus.

TF saturation models diminishing returns: In short technical abstracts, a term appearing 8 times is not 8 times more relevant evidence than one appearing once. The $k1 = 1.5$ parameter ensures that the marginal score contribution of each additional term occurrence decreases, producing more robust rankings than raw TF allows.

6.2 TF-IDF vs BM25

TF-IDF achieves MAP = 0.2169, which is 20.4% lower than BM25. The primary cause is the absence of document length normalisation. This is visible in Table 3: TF-IDF ranks irrelevant document 1268 first for Query 1 — a longer abstract that mentions query terms repeatedly without being a focused, relevant source. BM25 correctly relegates 1268 to rank 5 after accounting for its length.

It is notable that TF-IDF still outperforms Okapi TF (MAP = 0.2002) despite lacking length normalisation. This demonstrates that IDF weighting (present in TF-IDF but not Okapi TF) is more important than length normalisation alone for the Cranfield collection. The two components together — as in BM25 — produce the best results.

6.3 Okapi TF: Length Normalisation Without IDF

Okapi TF (MAP = 0.2002) performs worse than TF-IDF despite having length normalisation. This reveals a key insight: on the Cranfield collection, discriminating between common and rare terms (IDF) is more important than length normalisation. Okapi TF treats all query terms equally regardless of their collection frequency, meaning that a query containing the common term 'flow' and the rare term 'ablation' would assign equal importance to both — a poor approximation of true relevance.

This result strongly suggests that for specialised technical collections, IDF weighting is a prerequisite for good retrieval performance, not merely an enhancement.

6.4 Language Model Comparison: JM vs Laplace

The two language models show markedly different performance: Jelinek-Mercer achieves MAP = 0.2444 (second best overall), while Laplace achieves only MAP = 0.1070 (worst by a large margin). This gap is particularly striking because both models use the same query-likelihood framework with log-sum scoring; the difference lies entirely in how they smooth zero-probability terms.

Laplace smoothing adds exactly 1/(dl+V) to every term's probability, regardless of that term's actual frequency in the collection. This uniform smoothing fails to capture collection-level importance: a rare term like 'ablation' (with low collection frequency) and a common term like 'pressure' (with high collection frequency) both receive the same smoothing mass of approximately 0.000134. The model cannot distinguish discriminative from non-discriminative terms.

Jelinek-Mercer replaces this uniform mass with lambda x P(t|C), where P(t|C) = ctf(t)/C reflects actual collection term distribution. Terms common in the collection receive a higher floor probability; rare terms receive a lower one. This provides implicit IDF-like discrimination: documents that lack a rare query term receive a very small (but nonzero) probability for that term, effectively down-weighting those documents relative to ones that contain it. This explains why JM substantially outperforms Laplace on the Cranfield collection.

6.5 Per-Query Variability and Model Consistency

Table 2 reveals that the ranking of models is not entirely consistent across queries. Query 4 is a case where TF-IDF (AP = 0.6667) outperforms BM25 (AP = 0.5909). Examination of this query suggests that the relevant documents are unusually long, making raw TF accidentally more discriminative than the normalised BM25 score.

Query 9's high AP for both BM25 and Okapi TF (0.8056 each) illustrates cases where the relevant documents are easily separable by any TF-based model. The relevant documents likely use unique terminology that is strongly associated with the query topic and rarely appears in irrelevant documents.

The substantial variation in per-query AP across all models (from near-zero to over 0.8) highlights that no single model dominates in all cases. In production systems, query-adaptive scoring — adjusting model parameters based on query characteristics — could potentially improve performance across this variance.

7. Conclusion

This study implemented and systematically evaluated five classical IR ranking models on the Cranfield collection. The following conclusions are drawn from the experimental results.

BM25 is the strongest-performing model, achieving MAP = 0.2750 — the highest score across all five evaluated metrics. Its combination of Robertson IDF, length-normalised TF, and TF saturation makes it particularly well-suited to short technical document collections with precise queries. The Cranfield collection represents an ideal environment for BM25, and these results corroborate decades of IR literature establishing BM25 as the baseline against which other models are compared.

The Jelinek-Mercer language model (MAP = 0.2444) is competitive with TF-IDF and demonstrates that language modelling approaches can achieve strong performance when appropriate smoothing is applied. The interpolation with the collection model provides implicit IDF-like discrimination that makes JM robust on specialised technical collections.

The large performance gap between Jelinek-Mercer (MAP = 0.2444) and Laplace (MAP = 0.1070) is the most striking finding of this study. Both models use identical scoring frameworks; the difference is attributable entirely to the smoothing method. Laplace's uniform smoothing assigns equal importance to all unseen terms regardless of collection frequency, failing to discriminate between rare and common terms. This demonstrates that the choice of smoothing strategy in language models is more consequential than whether language modelling is used at all.

A comparison of TF-IDF (MAP = 0.2169) and Okapi TF (MAP = 0.2002) shows that IDF weighting is more important than length normalisation for the Cranfield collection. However, BM25 — which includes both — confirms that combining these components yields the best results.

Future work could explore: parameter optimisation for BM25 (k1 and b values via cross-validation); query expansion techniques to improve coverage of varied relevant vocabulary; and hybrid models that combine BM25 scoring with language model smoothing.

8. References

Cleverdon, C. W. (1967). The Cranfield tests on index language devices. Aslib Proceedings, 19(6), 173-194.

Robertson, S. E., & Sparck Jones, K. (1976). Relevance weighting of search terms. Journal of the American Society for Information Science, 27(3), 129-146.

Robertson, S. E., Walker, S., Jones, S., Hancock-Beaulieu, M. M., & Gatford, M. (1995). Okapi at TREC-3. In Proceedings of the Third Text Retrieval Conference (TREC-3). NIST Special Publication.

Ponte, J. M., & Croft, W. B. (1998). A language modeling approach to information retrieval. In Proceedings of the 21st Annual International ACM SIGIR Conference (pp. 275-281). ACM.

Zhai, C., & Lafferty, J. (2001). A study of smoothing methods for language models applied to ad hoc information retrieval. In Proceedings of the 24th Annual International ACM SIGIR Conference (pp. 334-342). ACM.

Manning, C. D., Raghavan, P., & Schutze, H. (2008). Introduction to Information Retrieval. Cambridge University Press.

Voorhees, E. M., & Harman, D. K. (Eds.) (2005). TREC: Experiment and Evaluation in Information Retrieval. MIT Press.

Sparck Jones, K. (1972). A statistical interpretation of term specificity and its application in retrieval. Journal of Documentation, 28(1), 11-21.